

Performing Sentiment Analysis With Naive Bayes Classifier

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
data = pd.read_csv('google_play_store_apps_reviews_training.csv')
#link for dataset: google\_play\_store\_apps\_reviews\_training.csv

print(data.head())
```

#Pre-process Data

```
def preprocess_data(data):
    # Remove package name as it's not relevant
    data = data.drop('package_name', axis=1)
    # Convert text to lowercase

    data['review'] = data['review'].str.strip().str.lower()
    return data
data = preprocess_data(data)
```

```
x = data['review']
```

```
y = data['polarity']
```

#Vectorize text reviews to numbers

```
# Vectorize text reviews to numbers

vec = CountVectorizer(stop_words='english')

x = vec.fit_transform(x).toarray()
```

Split into training and testing data

```
x_train, x_test, y_train, y_test = train_test_split(x,y, test_size=0.25,  
random_state=42)
```

#Model Generation

```
from sklearn.naive_bayes import MultinomialNB
```

```
model = MultinomialNB()
```

```
print(model.fit(x, y))
```

```
y_pred =model.predict(x_test)
```

```
model.score(x_test, y_test)
```

```
from sklearn.metrics import classification_report, confusion_matrix
```

```
print(confusion_matrix(y_test, y_pred))
```

```
print(classification_report(y_test, y_pred))
```

```
model.predict(vec.transform(['Love this app simply awesome!']))
```

#Predict for Custom Input:

```
def expression_check(prediction_input):
```

```
    if prediction_input == 0:
```

```
        print("Input statement has Negative Sentiment.")
```

```
    elif prediction_input == 1:
```

```
        print("Input statement has Positive Sentiment.")
```

```
    else:
```

```
        print("Invalid Statement.")
```

function to take the input statement and perform the same transformations we did earlier

```
def sentiment_predictor(input):  
    transformed_input = vec.transform(input)  
    prediction = model.predict(transformed_input)  
    expression_check(prediction)
```

```
input1 = ["Worst support. I have given 2 stars because changes to be done."]
```

```
input2 = ["Its working not nicely."]
```

```
sentiment_predictor(input1)
```

```
sentiment_predictor(input2)
```